

Introduction

This project simulates a complete OFDM wireless communication system. The simulation models the entire data pipeline from binary generation to signal modulation, transmission through a noisy channel, and final demodulation. To handle the random phase shifts like Rayleigh fading, the system relies on DPSK rather than standard QPSK. The system is built on OFDM design data in the project which are shown in the following section. This project contains five main functions which we will explain each of them in their own part.

1. The Transmitter (ofdmTransFrame)
2. The Receiver (ofdmRecFrame)
3. The Channel (addAWGN)
4. The Header Detector (detectHeaderStart)
5. The Main Loop (runOFDM)

```
clear; clc; close all;
nc      = 400;
Nfft    = 1024;
M        = 4;
cpRatio  = 0.25;
Ncp      = cpRatio*Nfft;
sfc      = ceil(2^13/nc);

rng('shuffle');
p.nc = nc; p.Nfft = Nfft; p.M = M; p.Ncp = Ncp; p.sfc = sfc;
p.frameLen = (sfc+1)*(Nfft+Ncp);
p.guardLen = Ncp;
p.headerLen = 8*p.frameLen;
p.header = sign(randn(1, p.headerLen));
p.frameGuard = zeros(1, p.guardLen);

numBits = 1e7;
bits     = randi([0 1], 1, numBits);
```

Part A

Part A serves as the baseline validation for the system architecture. In this section, 10 million random bits are generated and passed through the OFDM system over a AWGN channel. The fixed SNR in this part is 20 dB. Because our design allocates a significant portion of its power budget to Cyclic Prefix, wide zero-padded subcarrier bands, and massive synchronization headers, 20 dB represents a relatively low SNR. Therefore, detecting number of bits in this section is hard and mathematically expected and results demonstrate that the BER is very high. Also we can see other desired values like Length of frames, Length of header, Number of frames, Number of bit errors, Bit Error Rate.

```
[BER_a, numFrames_a, numErrors_a] = runOFDM(bits, p, 'awgn', 20);

fprintf('PART A RESULTS:\n');
```

PART A RESULTS:

```
fprintf('sfc = %d\n', sfc);
```

sfc = 21

```
fprintf('Length of frames = %d samples\n', p.frameLen);
```

Length of frames = 28160 samples

```
fprintf('Length of header = %d samples\n', p.headerLen);
```

Length of header = 225280 samples

```
fprintf('Number of frames = %d\n', numFrames_a);
```

Number of frames = 596

```
fprintf('Number of bit errors = %d\n', numErrors_a);
```

Number of bit errors = 4841199

```
fprintf('Bit Error Rate (BER) = %.6g\n\n', BER_a);
```

Bit Error Rate (BER) = 0.48412

Part BC

Parts B and C expand our simulation to evaluate the system's performance across a wide range of channel conditions. I combined sections B and C and compared both AWGN and RAYLEIGH channels. 10 million random bits are generated and passed through the system while we sweep the SNR from 0 dB to 50 dB over both cases. Rayleigh channel simulates real multipath environments where signals bounce off obstacles. Because Rayleigh fading introduces deep amplitude fades and random phase shifts, it is inherently more destructive than simple additive noise. Therefore, it is mathematically expected that the Rayleigh channel will require a significantly higher SNR to achieve the same low BER as the AWGN channel. Finally, the results of both sweeps are plotted together to show the performance gap and degradation caused by multipath fading against the AWGN channel.

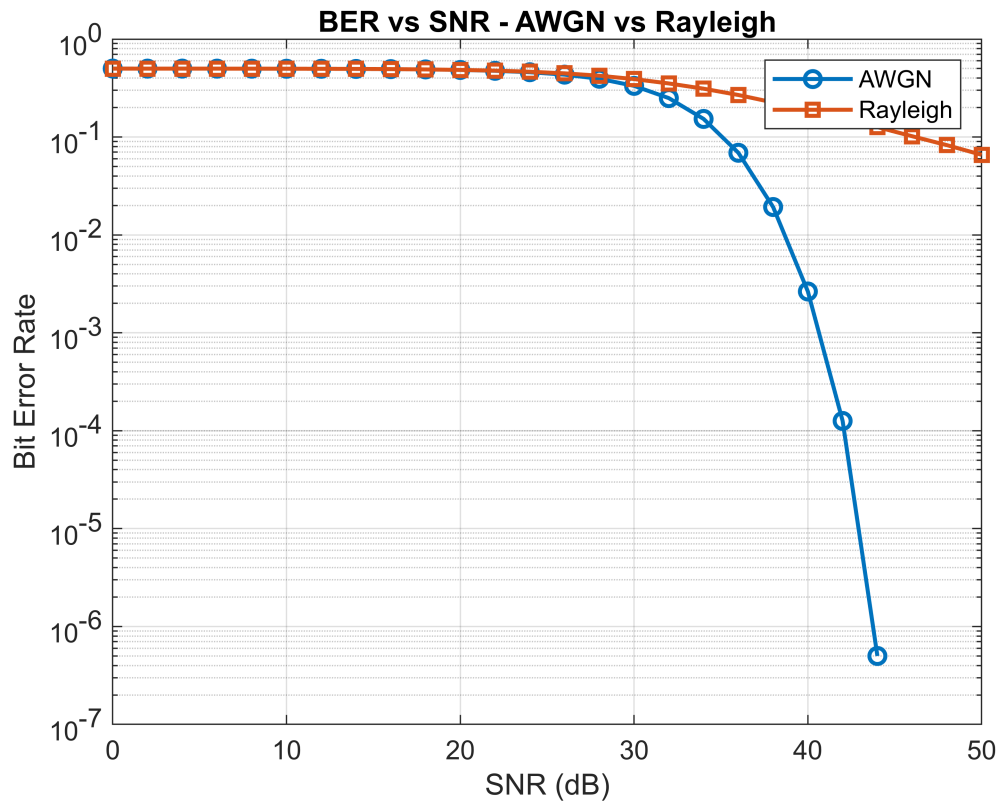
```
SNRlist = 0:2:50;

BER_awgn = zeros(size(SNRlist));
for i = 1:length(SNRlist)
    BER_awgn(i) = runOFDM(bits, p, 'awgn', SNRlist(i));
end

BER_rayleigh = zeros(size(SNRlist));
for i = 1:length(SNRlist)
    BER_rayleigh(i) = runOFDM(bits, p, 'rayleigh', SNRlist(i));
end

figure;
semilogy(SNRlist, BER_awgn, '-o', 'LineWidth', 1.5); hold on;
```

```
semilogy(SNRlist, BER_rayleigh, '-s', 'LineWidth', 1.5);
grid on; xlabel('SNR (dB)'); ylabel('Bit Error Rate');
legend('AWGN', 'Rayleigh');
title('BER vs SNR - AWGN vs Rayleigh');
```



Part D

In real-world communication systems, the receiver rarely knows the exact moment a transmission begins. Part D models this uncertainty by prepending a random amount of junk delay data to the beginning of the transmitted signal. Before the receiver can decode any data, it must first blindly locate where the actual frame starts. This is achieved using a synchronization block (detectHeaderStart) that will be explained.

```
[BER_d, numFrames_d, numErrors_d, delayLen_d, syncOK_d] = runOFDM(bits, p, 'awgn', 20, true);
fprintf('PART D RESULTS \n');
```

PART D RESULTS

```
fprintf('sfc = %d \n', sfc);
```

sfc = 21

```
fprintf('Length of frames = %d samples\n', p.frameLen);
```

Length of frames = 28160 samples

```
fprintf('Length of header      = %d samples\n', p.headerLen);
```

```
Length of header      = 225280 samples
```

```
fprintf('Number of frames      = %d\n', numFrames_d);
```

```
Number of frames      = 596
```

```
fprintf('Number of bit errors    = %d\n', numErrors_d);
```

```
Number of bit errors    = 4839783
```

```
fprintf('Bit Error Rate (BER)     = %.6g\n', BER_d);
```

```
Bit Error Rate (BER)     = 0.483978
```

```
fprintf('Random delay added samples = %d\n', delayLen_d);
```

```
Random delay added samples = 39957
```

```
fprintf('Correct Header  = %d\n\n', syncOK_d);
```

```
Correct Header  = 1
```

Part E

This section was added only for my personal visualization on how the channels physically distort our transmitted symbols and the constellation of Received signal. As seen in the plots, the AWGN channel simply adds background noise, scattering the data points neatly around their four ideal phase quadrants. In contrast, the Rayleigh fading channel changes symbol phases using random phase rotations and amplitude drops, which results into completely scattered points.

```
constSNRdB = 40;
```

```
frameLevelsExample = randi([0 M-1], nc*sfc, 1);  
[~, txSymbols] = ofdmTransFrame(frameLevelsExample, nc, sfc, Nfft, Ncp, M, []);
```

```
[frameSignal_awgn, ~] = ofdmTransFrame(frameLevelsExample, nc, sfc, Nfft, Ncp, M, []);  
noisySignal_awgn = addAWGN(frameSignal_awgn, constSNRdB, 1);  
[~, rxCarriers_awgn] = ofdmRecFrame(noisySignal_awgn, nc, sfc, Nfft, Ncp);
```

```
hVecExample = (randn(1,nc) + 1j*randn(1,nc))/sqrt(2);  
[frameSignal_ray, ~] = ofdmTransFrame(frameLevelsExample, nc, sfc, Nfft, Ncp, M, hVecExample);  
noisySignal_ray = addAWGN(frameSignal_ray, constSNRdB, 1);  
[~, rxCarriers_ray] = ofdmRecFrame(noisySignal_ray, nc, sfc, Nfft, Ncp);
```

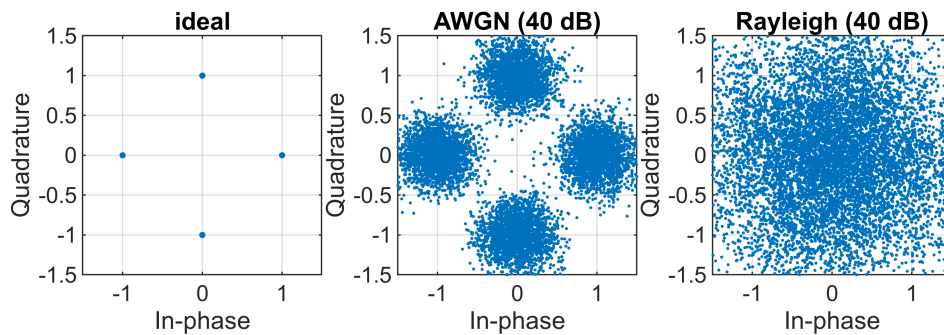
```
figure;  
subplot(1,3,1);  
plot(real(txSymbols(:)), imag(txSymbols(:)), '.', 'MarkerSize', 8);  
axis equal; grid on; xlim([-1.5 1.5]); ylim([-1.5 1.5]);  
xlabel('In-phase'); ylabel('Quadrature');  
title('ideal');
```

```

subplot(1,3,2);
plot(real(rxCarriers_awgn(:)), imag(rxCarriers_awgn(:)), '.', 'MarkerSize', 4);
axis equal; grid on; xlim([-1.5 1.5]); ylim([-1.5 1.5]);
xlabel('In-phase'); ylabel('Quadrature');
title(sprintf('AWGN (%d dB)', constSNRdB));

subplot(1,3,3);
plot(real(rxCarriers_ray(:)), imag(rxCarriers_ray(:)), '.', 'MarkerSize', 4);
axis equal; grid on; xlim([-1.5 1.5]); ylim([-1.5 1.5]);
xlabel('In-phase'); ylabel('Quadrature');
title(sprintf('Rayleigh (%d dB)', constSNRdB));

```



Transmitter frame

First, the serial to parallel block takes the incoming continuous stream of data and reshapes it into a two-dimensional matrix. A row of random reference symbols is immediately added to the very beginning of this grid. Because the system uses differential encoding, the receiver will need this initial arbitrary row.

Next, the differential modulation block processes the data iteratively through the matrix. We calculate the current phase by adding the new data value to the accumulated phase of the previous symbol and taking modulo of it. Then the mapping block converts these integer phase levels into complex numbers, placing them onto the unit circle as the constellation points. If we have a rayleigh fading vector it multiplies to the symbols vector.

Next part is IFFT block which maps data symbols to their assigned frequency. To ensure the final waveform is a real signal that can be transmitted, the function creates a mirrored, complex-conjugated copy of the data just like the the project requirements. Finally the CP will be added and reshaped to send in to the channel.

```

function [frameSignal, dpskSymbols] = ofdmTransFrame(frameLevels, nc, sfc, Nfft, Ncp, M, hVec)

% Serial to parallel
dataMat = reshape(frameLevels, sfc, nc);
refRow = randi([0 M-1], 1, nc);
fullMat = [refRow; dataMat];

% DPSK modulation
dpskLevels = zeros(size(fullMat));
dpskLevels(1,:) = fullMat(1,:); % reference row unchanged
for r = 2:size(fullMat,1)
    dpskLevels(r,:) = mod(dpskLevels(r-1,:) + fullMat(r,:), M);
end

% mapping
dpskSymbols = exp(1j*deg2rad(90*dpskLevels));

% if channel fading applied
if ~isempty(hVec)
    dpskSymbols = dpskSymbols .* hVec;
end

% IFFT
rows = size(dpskSymbols,1);
ifftMat = zeros(rows, Nfft);
ifftMat(:, 2:nc+1) = dpskSymbols; % DATA
ifftMat(:, Nfft-nc+1:Nfft) = conj(fliplr(dpskSymbols)); % DATA*
timeMat = real(ifft(ifftMat, Nfft, 2));

% CP
cpMat = [timeMat(:, end-Ncp+1:end), timeMat];

% Parallel to serial
frameSignal = reshape(cpMat.', 1, []);
end

```

Receiver frame

According to the system block diagram, we have these blocks in the receiver:

Frames Detection: This block processes the signal received from the channel to identify the transmitted frames.

Serial to Parallel: This block takes the continuous serial stream of incoming data and reshapes it into parallel matrices.

CP Removal: This step removes the CP that was added to the beginning of the symbols by the transmitter.

FFT: This block takes the FFT to convert the time-domain signal back into the frequency domain actually reversing the transmitter's IFFT operation.

Extract Carriers: After the FFT block, we should extract the subcarriers from the full set of FFT bins which the middle indices are zero that were added for guards and the finals are the conjugated frequencies so just from 2 to $nc+1$ is enough.

DPSK Demodulation: This block processes the extracted carriers to reverse the DPSK modulation that was applied at the transmitter which is done by taking mod of the phases.

Parallel to Serial: This block converts the parallel, demodulated data back into a single serial stream.

Cascade Frames: This block puts each serial frames together into a continuous sequence.

Estimation: The final block processes the sequence using a minimum error decision rule to output the original data.

```
function [rxLevels, rxCarriers] = ofdmRecFrame(segment, nc, sfc, Nfft, Ncp)

    % Serial to parallel
    rxMat = reshape(segment, Nfft+Ncp, sfc+1).';

    % CP removal
    rxNoCP = rxMat(:, Ncp+1:end);

    % FFT
    rxFreq = fft(rxNoCP, Nfft, 2);

    % Extract carriers from FFT bins
    rxCarriers = rxFreq(:, 2:nc+1);

    % DPSK demodulation (MDR)
    phase = mod(angle(rxCarriers), 2*pi);
    rxLevelsFull = mod(round(phase/(pi/2)), 4);

    % differential decoding
    dataLevelsHat = mod(rxLevelsFull(2:end,:) - rxLevelsFull(1:end-1,:), 4);

    % Parallel to serial
    rxLevels = dataLevelsHat(:).';
end
```

AWGN

The Additive White Gaussian Noise channel is the basic model used in communication systems.

Mathematically, this channel simply *adds* random, normally distributed noise values directly to the transmitted waveform. Because AWGN only adds random static without introducing the phase rotations in multipath environments, it is considered the standard for evaluating the system's BER.

```
function noisySignal = addAWGN(signal, SNRdB, sigma_s)

    SNRlin = 10^(SNRdB/10);
    sigma_n = sigma_s/sqrt(SNRlin);
    noisySignal = signal + sigma_n*randn(size(signal));
```

```
end
```

Header detection

This function is like a scanner looking for a specific header hidden in a stream of random junk data. It slides a clean copy of the header over the incoming signal. When they match up perfectly, it creates a massive mathematical spike, telling the receiver exactly where the real data begins.

The search limit is just a smart shortcut. We know the random junk data will never be longer than 50,000 samples (This is in the next part). So, instead of scanning the entire signal, we only check the very beginning. This makes the code run almost instantly and stops it from accidentally locking onto the second header located at the very end of the transmission.

```
function startIdx = detectHeaderStart(rxSignal, header)

    search_limit = min(length(rxSignal), length(header) + 100000);
    search_window = rxSignal(1 : search_limit);

    [corr_out, lags] = xcorr(search_window, header);
    [~, peak_idx] = max(abs(corr_out));
    estimated_delay = lags(peak_idx);

    if estimated_delay < 0
        startIdx = 1;
    else
        startIdx = estimated_delay + 1;
    end

end
```

Run OFDM

The runOFDM function is the master control center for the entire simulation. Instead of running each step manually, you simply feed this function your raw data bits and the desired channel conditions. It automatically routes signal through the transmitter, simulating the real-world channel noise, recovering the data in the receiver, and finally comparing the output against your original input to calculate exactly how many errors occurred along the way.

It Pairs the incoming stream of binary bits and converts them into the initial integer data levels (0, 1, 2, or 3) by adding each two consecutive bits. Then adding the zero-padding levels and reshape the last frame. Afterwards, it will add a random noise into the transmitted signal and the rayleigh multiplication vector if needed. Then uses the header detection block to scan the noisy signal and locate the start of the true data. We set a pointer which points to the start of the main signal(without delay and header) to send to receiver and make the frame. Finally, it Maps the recovered data levels back into raw binary bits and compares them against the original input to calculate the final BER.

```
function [BER, numFrames, numErrors, delayLen, syncOK] = runOFDM(bits, p, channelType, SNRdB, c
```



```

if nargin < 5
    delayFlag = false;
end

numBits = length(bits);

% QPSK
b1 = bits(1:2:end);
b2 = bits(2:2:end);
levels = b1*2 + b2;
numSym = length(levels);

% Frame divider + Serial to Parallel
frameCap = p.nc*p.sfc;
numFrames = ceil(numSym/frameCap);
padLen = numFrames*frameCap - numSym;
levelsPadded = [levels, zeros(1, padLen)];
allFrames = reshape(levelsPadded, frameCap, numFrames);

% Random Delay
if delayFlag
    delayLen = randi([1000, 50000]);
    delayData = randn(1, delayLen);
else
    delayLen = 0;
    delayData = [];
end

% build & cascade
txSignal = [delayData, p.header];
for f = 1:numFrames
    frameLevels = allFrames(:, f);

    if strcmpi(channelType, 'rayleigh')
        hVec = (randn(1,p.nc) + 1j*randn(1,p.nc))/sqrt(2);
    else
        hVec = [];
    end

    frameSignal = ofdmTransFrame(frameLevels, p.nc, p.sfc, p.Nfft, p.Ncp, p.M, hVec);
    txSignal = [txSignal, frameSignal, p.frameGuard]; %#ok<AGROW>
end

txSignal = [txSignal, p.header];
sigma_s = 1;
rxSignal = addAWGN(txSignal, SNRdB, sigma_s);

if delayFlag
    startIdx = detectHeaderStart(rxSignal, p.header);

```

```

        syncOK    = (startIdx == delayLen + 1);
else
    startIdx = 1;
    syncOK    = true;
end
ptr = (startIdx - 1) + p.headerLen;

% Receiver
rxLevelsAll = [];
for f = 1:numFrames
    segment = rxSignal(ptr+1 : ptr+p.frameLen);
    ptr = ptr + p.frameLen + p.guardLen;
    frameLevelsHat = ofdmRecFrame(segment, p.nc, p.sfc, p.Nfft, p.Ncp);
    rxLevelsAll = [rxLevelsAll, frameLevelsHat]; %#ok<AGROW>
end
rxLevelsAll = rxLevelsAll(1:numSym);

% QPSK demodulation
rb1 = floor(rxLevelsAll/2);
rb2 = mod(rxLevelsAll, 2);
rxBits = zeros(1, numBits);
rxBits(1:2:end) = rb1;
rxBits(2:2:end) = rb2;

numErrors = sum(rxBits ~= bits);
BER = numErrors/numBits;
end

```